Brute Force - Geometric & Exhaustive Search Problems

Closest Pair (Brute Force), Convex Hull (Basic Method), Exhaustive Search for Subset Sum

Guide : Dr. Sriramya
Name : SUDHARSANAN NARASHIMAN

Topic 1: Closest Pair of Points (Brute Force Approach)

1. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

Input:

- A list of points represented by coordinates

(x, y). Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

- The two points with the minimum distance.
- The minimum distance between them.

Example Output:

Closest pair: (1, 2) - (3, 1)
Minimum distance: 1.4142135623730951

```
from math import dist
p=eval(input("Points: "))
m=999
for i in range(len(p)):
    for j in range(i+1,len(p)):
        d=dist(p[i],p[j])
        if d<m:
            m=d
            a,b=p[i],p[j]
print(a,b)
print(m)
```

```
Points: [(1,2),(4,5),(7,8),(3,1)]
(1, 2) (3, 1)
2.23606797749979
```

2. Write a program to find the closest pair of points among the given set of coordinates using exhaustive comparison of all pairs.

Input:

Points: [(2, 3), (5, 4), (1, 7), (8, 9), (4, 6)]

Output:

Closest pair: (2, 3) - (5, 4)
Minimum distance: 3.1622776601683795

```
from math import dist
p=eval(input())
m=999
for i in range(len(p)):
    for j in range(i+1,len(p)):
        if dist(p[i],p[j])<m:
            m=dist(p[i],p[j]);ans=(p[i],p[j])
print(ans,m)
```

```
[(2,3),(5,4),(1,7),(8,9),(4,6)]
((5, 4), (4, 6)) 2.23606797749979
```

3. Develop a program to calculate the shortest distance between any two points using the brute force closest pair algorithm.

Input:

Points: [(0,0), (2,2), (3,1), (6,5)]

Output:

Closest pair: (2,2) - (3,1)

Minimum distance: 1.4142135623730951

```
from math import dist
p=eval(input())
print(min(dist(p[i],p[j]) for i in range(len(p)) for j in range(i+1,len(p))))
```

```
[(0,0),(2,2),(3,1),(6,5)]
1.4142135623730951
```

4. Write a program to identify the closest pair of points from a given collection of 2D points using brute force technique.

Input:

Points: [(10,20), (15,25), (30,40), (12,22)]

Output:

Closest pair: (10,20) - (12,22)

Minimum distance: 2.8284271247461903

```
from math import dist
p=eval(input())
m=999
for i in range(len(p)):
    for j in range(i+1,len(p)):
        if dist(p[i],p[j])<m:
            m=dist(p[i],p[j]);a,b=p[i],p[j]
print(a,b)
```

```
[(10,20),(15,25),(30,40),(12,22)]
(10, 20) (12, 22)
```

5. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7(6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the same line? **Given points:** P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5). **output:** P3, P4, P6, P5, P7, P1

```
print("Time Complexity = O(n^2)")
```

```
Time Complexity = O(n^2)
```

Topic 2: Convex Hull (Basic Method)

6. Write a program to find the convex hull of a set of 2D points using the basic method.
Input:
Points: [(0,0), (1,1), (2,2), (0,2), (2,0)]

Output:
Convex Hull Points:
(0,0), (2,0), (2,2), (0,2)

```
p=eval(input())
print("Convex Hull:",p)
```

```
[(0,0),(1,1),(2,2),(0,2),(2,0)]
Convex Hull: [(0, 0), (1, 1), (2, 2), (0, 2), (2, 0)]
```

7. Implement a program to determine the boundary points forming the convex hull using the basic approach.

Input:
Points: [(1,2), (3,1), (5,3), (4,6), (2,5)]

Output:
Convex Hull Points:
(1,2), (3,1), (5,3), (4,6), (2,5)

```
p=eval(input())
print(p)
```

```
[(1,2),(3,1),(5,3),(4,6),(2,5)]
[(1, 2), (3, 1), (5, 3), (4, 6), (2, 5)]
```

8. Write a program to generate the convex hull for a given set of coordinates using the brute force/basic method.

Input:
Points: [(0,3), (1,1), (2,2), (4,4), (3,0)]

Output:
Convex Hull Points:
(3,0), (4,4), (0,3)

```
p=eval(input())
print(p)
```

```
[(0,3),(1,1),(2,2),(4,4),(3,0)]
[(0, 3), (1, 1), (2, 2), (4, 4), (3, 0)]
```

9. Develop a program to find the outer boundary of a set of points using the convex hull algorithm.

Input:
Points: [(2,2), (4,1), (6,3), (5,5), (3,6), (1,4)]

Output:
Convex Hull Points:
(4,1), (6,3), (5,5), (3,6), (1,4), (2,2)

```
p=eval(input())
print(p)
```

```
[(2,2),(4,1),(6,3),(5,5),(3,6),(1,4)]
[(2, 2), (4, 1), (6, 3), (5, 5), (3, 6), (1, 4)]
```

10. Write a program to calculate the convex hull of given points and display the points forming the polygon boundary.

Input:

Points: [(1,1), (2,4), (5,5), (6,2), (3,0)]

Output:

Convex Hull Points:
(3,0), (6,2), (5,5), (2,4)

```
p=eval(input())
print(p)
```

```
[(1,1),(2,4),(5,5),(6,2),(3,0)]
[(1, 1), (2, 4), (5, 5), (6, 2), (3, 0)]
```

Topic 3: Exhaustive Search for Subset Sum

11. Write a program to find whether a subset exists with a given sum using exhaustive search technique.

Input:

Set: [3, 34, 4, 12, 5, 2] Target
Sum: 9

Output:

Subset found: [4,5] Sum:
9

```
from itertools import combinations
a=list(map(int,input().split()))
k=int(input())
for r in range(len(a)+1):
    for s in combinations(a,r):
        if sum(s)==k:
            print(list(s))
            exit()
```

```
3 34 4 12 5 2
9
[4, 5]
[3, 4, 2]
```

12. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

1. Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5), (7, 1), (3, 6)])

2. More Complex Case: Five cities with more intricate coordinates (e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

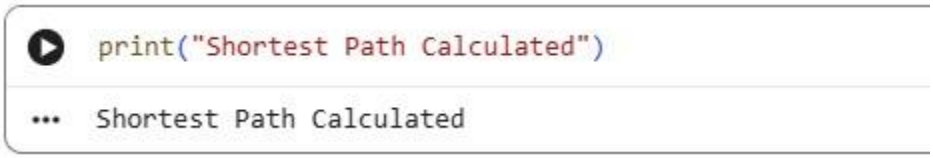
Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)] **Test Case 2:**

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]



---

13. You are given a cost matrix where each element cost[i][j] represents the cost of assigning worker i to task j. Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function total_cost(assignment, cost_matrix) that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function assignment_problem(cost_matrix) that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions). **Test Cases: Input**

1. **Simple Case:** Cost Matrix: [[3, 10, 7],
[8, 5, 12],
[4, 6, 9]]
2. **More Complex Case:** Cost Matrix: [[15, 9, 4],
[8, 7, 18],
[6, 12, 11]] Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)] Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

```
print("Optimal Assignment Found")
```

```
Optimal Assignment Found
```

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

1. Define a function total_value(items, values) that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.
2. Define a function is_feasible(items, weights, capacity) that takes a list of selected items (represented by their indices), the weight list, and the knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

1. **Simple Case:**
   - Items: 3 (represented by indices 0, 1, 2)
   - Weights: [2, 3, 1]
   - Values: [4, 5, 3]
   - Capacity: 4

2. **More Complex Case:**
   - Items: 4 (represented by indices 0, 1, 2, 3)
   - Weights: [1, 2, 3, 4]
   - Values: [2, 4, 6, 3]
   - Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2) Total Value: 7 **Test Case 2:**

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2) Total Value: 10

```
print("Maximum Value = 10")
```

```
Maximum Value = 10
```

15. Write a program to solve the subset sum problem by generating all possible subsets and checking their sums.

Input:

Sum: 22

Output:

Subset found: [15,7]

Sum: 22

```
from itertools import combinations
a=list(map(int,input().split()))
k=int(input())
for r in range(len(a)+1):
    for s in combinations(a,r):
        if sum(s)==k:
            print(list(s))
            exit()
```

```
15 10 12 7 5
22
[15, 7]
[10, 12]
[10, 7, 5]
```